

Étape 2

Organisation des données

Deux bases, une couche brute et une couche exploitable, reliées par une clé métier

Projet	CryptoBot, bot de trading piloté par le machine learning
Formation	Data Engineer, DataScientest
Dépôt	DataScientest-Studio/jul26_bde_crypto
Date	2 octobre 2026

Sommaire

1. Objectif de l'étape

2. Nos choix en bref

3. Notions de base

3.1 Base relationnelle et base orientée documents

3.2 Séries temporelles et TimescaleDB

3.3 Clé métier

4. Choix des bases

4.1 Dimensionner avant de choisir

4.2 Solutions évaluées

4.3 Critère de répartition : la forme de la donnée

4.4 Deux couches

5. Modèle de données

5.1 Une table de faits par profil

5.2 Schéma

5.3 Contraintes d'intégrité

5.4 La couche brute dans MongoDB

6. Relier les deux bases

7. Chargement des données

7.1 Principe : un chargement idempotent

7.2 Étapes

8. Évolution du modèle pendant le projet

9. Consultation des données

10. Annexe : fichiers

1. Objectif de l'étape

L'étape 2 consiste à choisir où stocker les données collectées, à concevoir le modèle de données et à écrire le chargement. Elle demande de justifier les choix : pourquoi ces bases plutôt que d'autres, et comment elles se relient.

Demande	Réalisation	Emplacement
Choix des bases de données	PostgreSQL avec TimescaleDB, et MongoDB	<code>docker-compose.yml</code>
Modèle de données (UML)	tables, clés, relations, collections	section 4 et <code>sql/01_schema.sql</code>
Chargement des données	fichiers de l'étape 1 vers les deux bases	<code>scripts/load_to_db.py</code>
Contrôle de l'état des bases	volumes, intégrité, retard de collecte	<code>scripts/check_db.py</code>

2. Nos choix en bref

Choix	Pourquoi	Alternative écartée
Répartir selon la forme de la donnée	une bougie a toujours 13 colonnes : relationnel ; les métadonnées de Binance ont des champs variables et imbriqués : documents	« SQL pour l'historique, NoSQL pour le temps réel », alors que la bougie est la même
PostgreSQL pour la couche exploitable	transactions, jointures, contraintes qui refusent une donnée incohérente ; 2 millions de lignes restent loin de ses limites	InfluxDB, ClickHouse : faits pour des milliards de lignes, sans transactions
Extension TimescaleDB	découpe les bougies en tranches de temps, compresse les anciennes ; fournie dans l'image Docker officielle	Snowflake, BigQuery : payants et hébergés, hors conteneurs
MongoDB pour la couche brute	garde les réponses de Binance telles quelles, pour rejouer un nettoyage corrigé	Elasticsearch, conçu pour la recherche
Clé métier (paire, pas de temps, ouverture)	identifie une bougie par ce qui la définit ; aucun doublon possible	un identifiant technique
Champ <code>raw_ref</code> sur chaque bougie	aucun moteur ne pose de clé étrangère vers l'autre ; on remonte ainsi au document brut d'origine	deux bases sans lien vérifiable
Une table par profil, lue par des vues	chaque pas de temps stocké une fois ; compression réglée par profil	une table unique
Chargement idempotent	une bougie présente est mise à jour, jamais dupliquée : relancer ne crée aucun doublon	une insertion simple

Les variables calculées à l'étape 3 sont stockées en JSONB plutôt qu'en colonnes : leur nombre a changé plusieurs fois, et chaque essai aurait demandé une migration du schéma.

3. Notions de base

3.1 Base relationnelle et base orientée documents

Une **base relationnelle** (PostgreSQL) range les données en tables de colonnes fixes et typées. Le schéma est déclaré à l'avance ; la base refuse une ligne qui ne le respecte pas. Elle garantit les transactions ACID : une écriture est entièrement faite ou pas du tout, même en cas de panne au milieu. Elle permet les jointures entre tables et les contraintes d'intégrité (clé primaire, clé étrangère, contrôle de valeur).

Une **base orientée documents** (MongoDB) range des documents JSON dans des collections. Deux documents d'une même collection peuvent avoir des champs différents, et un champ peut contenir des listes ou des sous-documents. On y stocke un objet tel qu'il arrive, sans le découper au préalable.

Le choix entre les deux dépend de la forme des données plus que de leur volume.

3.2 Séries temporelles et TimescaleDB

Nos données sont des **séries temporelles** : chaque ligne est horodatée, arrive dans l'ordre du temps et n'est plus modifiée ensuite. On les lit presque toujours par période (« les bougies de BTC en 1h depuis juin »).

TimescaleDB est une extension de PostgreSQL conçue pour ce cas. Elle découpe une table en tranches de temps appelées *chunks* ; on parle alors d'**hypertable**. Une requête sur une période ne lit que les tranches concernées, les anciennes tranches peuvent être compressées, et supprimer des données anciennes revient à supprimer des tranches entières plutôt qu'à effacer des lignes une à une. Comme il s'agit d'une extension, tout le reste de PostgreSQL (SQL, psycopg2, pgAdmin) fonctionne sans changement.

3.3 Clé métier

Une **clé métier** identifie un objet par ce qui le définit dans le domaine, et non par un numéro technique. Une bougie est définie par sa paire, son pas de temps et son heure d'ouverture : le triplet (`symbol`, `interval`, `open_time`) est sa clé métier. Il ne peut exister qu'une bougie BTCUSDT 1h ouverte le 28 août 2026 à midi.

4. Choix des bases

4.1 Dimensionner avant de choisir

À l'issue de l'étape 1, il faut stocker 2 075 570 bougies, soit 132 Mo. PostgreSQL commence à ralentir autour de quelques dizaines de millions de lignes, quand ses index ne tiennent plus en mémoire : nous en sommes loin. Les arguments de performance brute ne sont donc pas déterminants ici. Le critère retenu est la forme des données.

4.2 Solutions évaluées

Solution	Points forts	Points faibles	Décision
PostgreSQL	transactions ACID, jointures, contraintes, bien connu de l'équipe	pas de fonctions temporelles avancées sans extension	retenu
TimescaleDB	hypertables, compression, politiques de rétention par table	une extension de plus à installer	retenu (sur PostgreSQL)
MongoDB	schéma libre, stocke le JSON des API tel quel	jointures coûteuses	retenu
Elasticsearch	recherche plein texte, agrégations rapides	conçu pour la recherche, pas pour du stockage transactionnel	écarté
InfluxDB, ClickHouse	ingestion et agrégation très rapides	dimensionnés pour des milliards de lignes, sans ACID	écarté
Snowflake, BigQuery	aucune administration	payants et hébergés : incompatibles avec la conteneurisation demandée à l'étape 4	écarté

La première version du dossier de l'étape 2 reportait TimescaleDB à plus tard. Nous l'avons finalement adopté dès la mise en place : l'image Docker officielle fournit PostgreSQL avec l'extension déjà installée, sans coût d'exploitation supplémentaire, et la compression par table s'accorde avec le découpage en profils décrit en section 4.

4.3 Critère de répartition : la forme de la donnée

Un découpage fréquent sépare « SQL pour l'historique, NoSQL pour le temps réel ». Nous l'avons écarté : il sépare par provenance, alors que l'étape 1 a montré qu'une bougie reçue par WebSocket est exactement la même chose qu'une bougie reçue par l'API REST.

Nous répartissons selon la forme :

- **structure stable et connue à l'avance** : relationnel. Une bougie a toujours les mêmes 13 colonnes et une clé métier sans doublon.
- **structure variable ou imbriquée** : documents. Les métadonnées de Binance (`exchangeInfo`) contiennent une liste de filtres dont chaque type a des champs différents :

```
PRICE_FILTER  -> filterType, minPrice, maxPrice, tickSize
LOT_SIZE      -> filterType, minQty, maxQty, stepSize
NOTIONAL      -> filterType, minNotional, applyToMarket, avgPriceMins
```

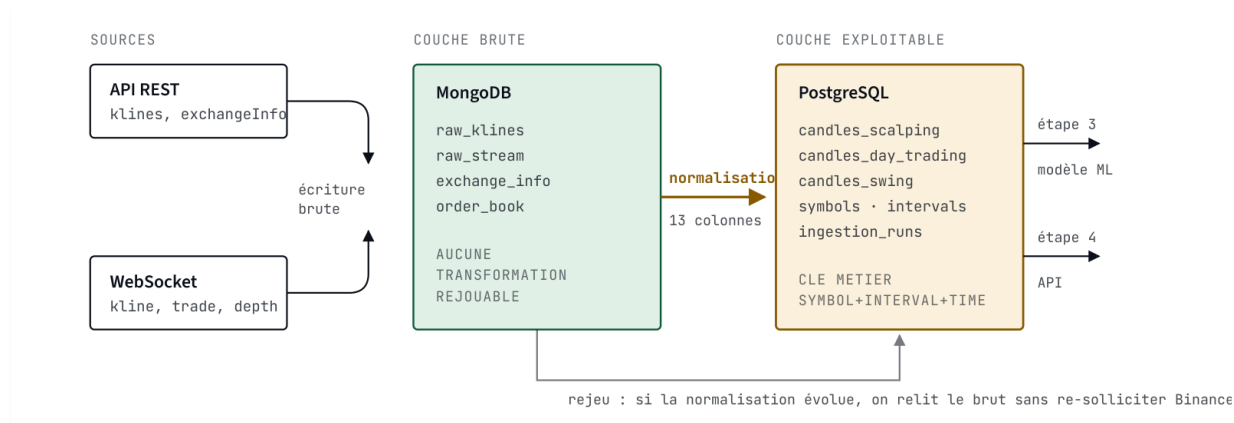
En relationnel, il faudrait une table par type de filtre ou une table générique clé-valeur, qui rend toute requête illisible. En document, c'est une liste de sous-documents.

4.4 Deux couches

Cette répartition donne deux couches aux rôles distincts.

Couche	Base	Contenu	Rôle
Brute	MongoDB	réponses de Binance telles quelles, métadonnées, messages WebSocket, carnets d'ordres	conserver l'original ; permettre de rejouer un nettoyage corrigé
Exploitable	PostgreSQL + TimescaleDB	bougies nettoyées, référentiels, journaux, variables, résultats	alimenter le modèle, l'API et la supervision

Les deux dossiers de l'étape 1 correspondent déjà à ces deux couches : `data/raw/` devient MongoDB, `data/processed/` devient PostgreSQL.



Flux des données : les sources écrivent le brut dans MongoDB, la normalisation alimente PostgreSQL

5. Modèle de données

5.1 Une table de faits par profil

Chaque profil de trading (scalping, day trading, swing) a sa propre table de bougies : `candles_scalping`, `candles_day_trading`, `candles_swing`. Les trois ont une structure identique.

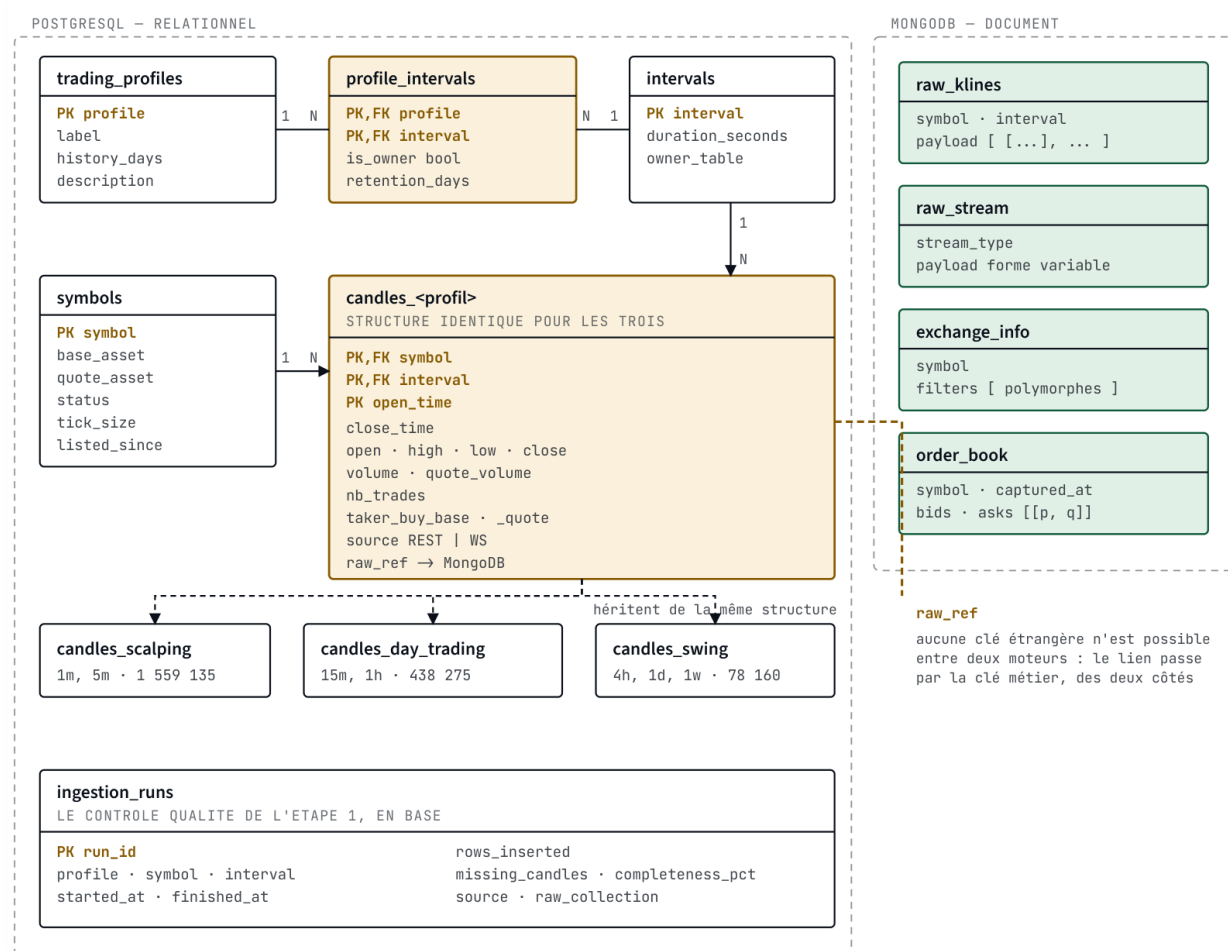
Deux pas de temps sont utilisés par deux profils : le 15m (scalping et day trading) et le 4h (day trading et swing). Les stocker deux fois dupliquerait les périodes qui se recouvrent. La règle appliquée est celle de la collecte : **chaque pas de temps est stocké une seule fois, dans la table du profil qui le conserve le plus longtemps**. Les autres profils le lisent par une vue.

Profil	Pas de temps utilisés	Stockés dans sa table	Lu dans une autre table
Scalping	1m, 5m, 15m	1m, 5m	15m (day trading)
Day trading	15m, 1h, 4h	15m, 1h	4h (swing)
Swing	4h, 1d, 1w	4h, 1d, 1w	aucun

Les vues `v_scalping`, `v_day_trading` et `v_swing` restituent à chaque profil ses trois pas de temps. Le code qui lit les données (modèle, API) interroge toujours une vue et ignore dans quelle table physique se trouve chaque bougie.

Ce découpage a un second intérêt : les politiques de TimescaleDB s'appliquent par table, donc par profil. Les tranches de temps sont de 7 jours pour le scalping (données denses), 30 jours pour le day trading et 365 jours pour le swing. Les tranches anciennes sont compressées (après 14, 90 et 365 jours selon le profil) : une bougie clôturée ne change plus, ses tranches sont en lecture seule.

5.2 Schéma



Modèle de données : référentiels, tables de faits, journal des collectes et collections MongoDB

Le référentiel est en haut, les faits au centre, le journal des collectes en bas. Les quatre collections MongoDB sont à droite.

Table	Rôle
trading_profiles	les trois profils et leur profondeur d'historique
intervals	les pas de temps, leur durée en secondes et la table qui les stocke
profile_intervals	quel profil utilise quel pas de temps, et s'il le stocke (<code>is_owner</code>)
symbols	les paires, leurs actifs et leur pas de cotation
candles_<profil>	les bougies, avec leur référence vers le document brut
ingestion_runs	une ligne par chargement : période, lignes, complétude, statut

Le champ `is_owner` inscrit la règle de stockage dans le modèle lui-même : la ligne (scalping, 15m, `is_owner = false`) signifie que le scalping utilise le 15m sans le stocker. Changer la répartition ne demande pas de modifier le code.

5.3 Contraintes d'intégrité

La base refuse les données incohérentes plutôt que de les accepter en silence :

Contrainte	Règle
clé primaire	(<code>symbol</code> , <code>interval</code> , <code>open_time</code>) : pas de doublon possible
clés étrangères	la paire et le pas de temps existent dans les référentiels
<code>ohlc_coherent</code>	le plus haut est au-dessus de l'ouverture, de la clôture et du plus bas
<code>volumes_positifs</code>	aucun volume négatif
<code>bougie_ordonnee</code>	la clôture est postérieure à l'ouverture
<code>source_connue</code>	la source vaut <code>REST</code> ou <code>WS</code>

Un index (`symbol` , `interval` , `open_time` décroissant) sert la lecture la plus fréquente : une paire, un pas de temps, sur une période ou pour les N dernières bougies.

5.4 La couche brute dans MongoDB

Collection	Contenu	Index
<code>raw_klines</code>	réponses de l'API de bougies, par lots de 1 000	unique sur (paire, pas de temps, numéro de lot) ; bornes temporelles
<code>raw_stream</code>	messages WebSocket de tous types	type de flux, paire, date de réception ; index partiel limité aux bougies clôturées
<code>exchange_info</code>	métadonnées des paires et leurs filtres	paire, date de collecte
<code>order_book</code>	instantanés du carnet d'ordres	paire, date de capture ; suppression automatique après 30 jours

Deux mécanismes propres à MongoDB sont utilisés. Un **index partiel** ne référence que les documents qui vérifient une condition : ici, les seules bougies clôturées du flux temps réel, les seules que l'on relit. Un **index TTL** (*time to live*) supprime automatiquement les documents plus anciens qu'une durée donnée : les instantanés du carnet d'ordres, volumineux, sont effacés après 30 jours sans traitement de purge.

Un document de `raw_klines` contient 1 000 bougies, soit exactement une réponse de l'API. Ce découpage suit la structure réelle de la donnée, et il est nécessaire : un document MongoDB est limité à 16 Mo, alors qu'un fichier de bougies d'une minute en fait 45.

6. Relier les deux bases

Aucun moteur ne peut imposer une clé étrangère vers l'autre. Le lien repose sur la clé métier, présente des deux côtés, et sur une référence explicite : chaque bougie de PostgreSQL porte un champ `raw_ref` qui désigne le document MongoDB dont elle vient.

```
PostgreSQL  symbol = 'BTCUSDT', interval = '1h', open_time = '2026-08-28 12:00 UTC'
              raw_ref = 'raw_klines/BTCUSDT/1h/...'
MongoDB      { symbol: 'BTCUSDT', interval: '1h', payload: [[1787918400000, '79604.12',
...]] }
```

Depuis n'importe quelle ligne, on remonte ainsi à la réponse exacte de Binance qui l'a produite. Si une valeur paraît suspecte, on vérifie l'original au lieu de supposer.

7. Chargement des données

7.1 Principe : un chargement idempotent

Un traitement est **idempotent** s'il donne le même résultat qu'on le lance une ou plusieurs fois. Le chargement l'est : une bougie déjà présente est mise à jour, jamais dupliquée (`INSERT ... ON CONFLICT DO UPDATE` sur la clé métier). Relancer un chargement interrompu ne crée donc aucun doublon, ce qui a rendu possible l'automatisation de l'étape 5.

7.2 Étapes

1. Écrire les réponses brutes dans MongoDB, sans aucune transformation.
2. Écrire les bougies nettoyées dans la table du profil propriétaire du pas de temps, avec leur `raw_ref`.
3. Consigner le chargement dans `ingestion_runs` : période, lignes lues et stockées, statut.

Après une écriture groupée (`execute_values`), le nombre de lignes renvoyé par le pilote ne compte que le dernier lot envoyé : sur 70 124 lignes, il annonçait 124. Le journal interroge donc la table pour connaître le nombre réellement stocké.

```
docker compose up -d          # démarre les deux bases
python -m scripts.load_to_db --check  # vérifie les connexions
python -m scripts.load_to_db        # charge les 35 jeux de données
python -m scripts.check_db          # volumes, intégrité, retard
```

8. Évolution du modèle pendant le projet

Le modèle a accueilli sans refonte les besoins des étapes suivantes. Chaque ajout est un fichier SQL numéroté, joué dans l'ordre à la création de la base.

Fichier	Ajout	Étape
05_features.sql	technical_features : les variables calculées pour chaque bougie	3
06_api.sql	api_predictions : chaque prédiction servie par l'API	4
07_positions.sql	positions_virtuelles : le carnet de trades simulés	4
08_suivi.sql	carnet_suivi : dernière bougie évaluée par le bot	4
09_pipeline.sql	contrôles qualité, mesures de dérive, réentraînements, compte de lecture pour Grafana	5

Les variables techniques sont stockées en JSONB (un champ JSON indexable de PostgreSQL) plutôt qu'en colonnes fixes. Le jeu de variables a changé plusieurs fois (26 puis 35 variables) : avec des colonnes, chaque essai aurait demandé une migration du schéma, et deux jeux n'auraient pas pu cohabiter. La recette qui a produit chaque jeu (familles d'indicateurs, fenêtres, version) est stockée dans MongoDB (feature_configs), conformément au critère de forme : c'est un document dont les champs varient d'un jeu à l'autre.

9. Consultation des données

Deux interfaces web permettent de parcourir les bases, pour une démonstration ou une vérification : pgAdmin pour PostgreSQL (localhost:5050) et mongo-express pour MongoDB (localhost:8081). Aucun traitement n'en dépend, et elles n'écotent que sur la machine locale.

10. Annexe : fichiers

Fichier	Rôle
sql/01_schema.sql	tables, contraintes, hypertables, index
sql/02_seed.sql	profils, pas de temps, règle de propriété
sql/03_views.sql	vues par profil et vue de couverture
sql/04_policies.sql	compression par profil
mongo/01_init.js	collections et index MongoDB
scripts/load_to_db.py	chargement des fichiers vers les deux bases
scripts/check_db.py	état des bases